Unreal Horde Agent Deployment (Docker)

Hidde Derks
02 Dec 2025 — 4 min read

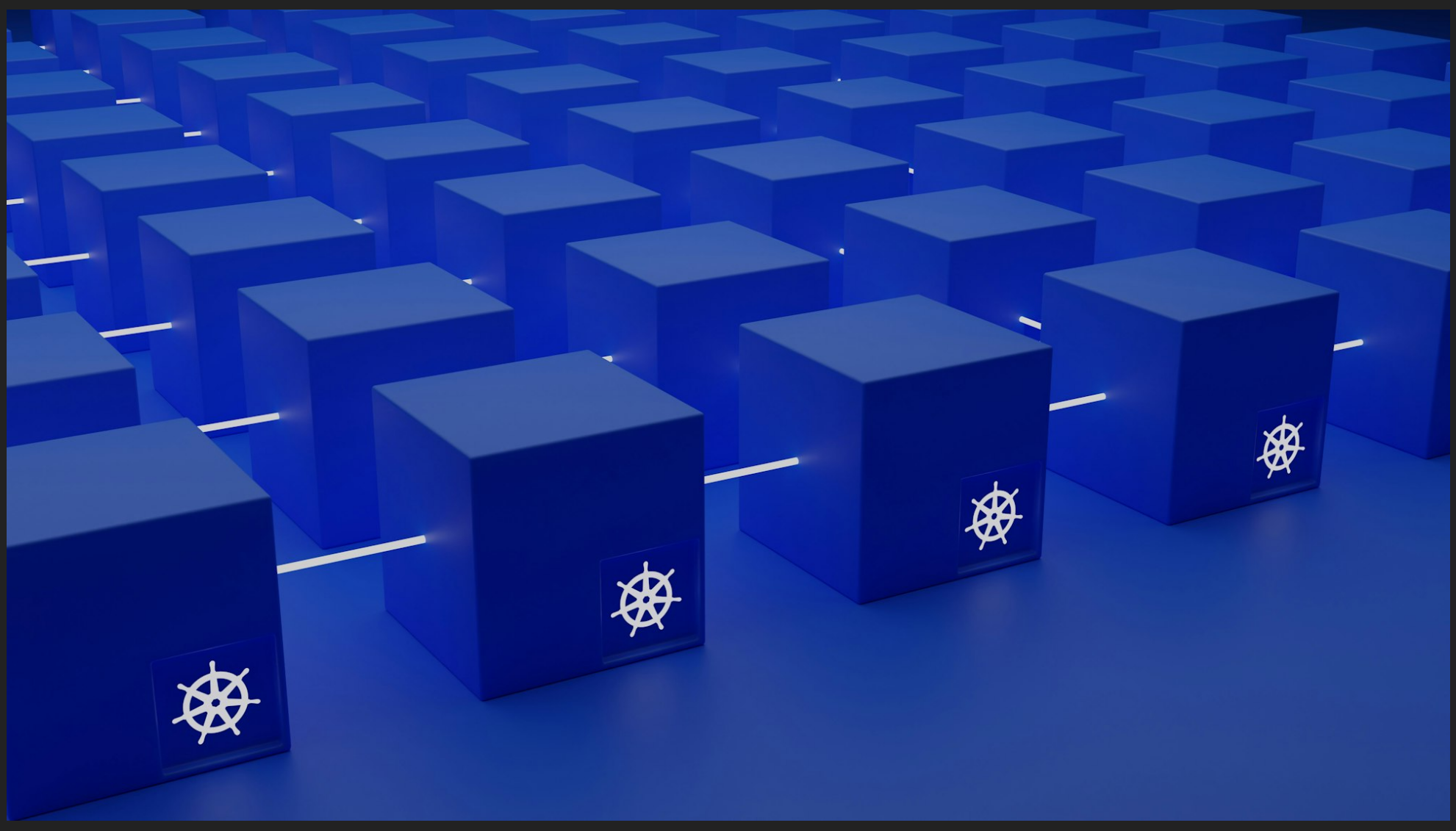


Photo by Growtika / Unsplash

> Initially created for the Breda University of Applied Sciences Master's of Game Technologies

To start benefiting from the Unreal Build Accelerator you need to have a server set up for agents to connect to. In case you're looking for server set up, choose one of the following:

Horde Server (Windows)

Horde Server (Docker)

This guide will focus on deploying the Horde Agent in Docker, if you're looking for the Windows deployment, go here:

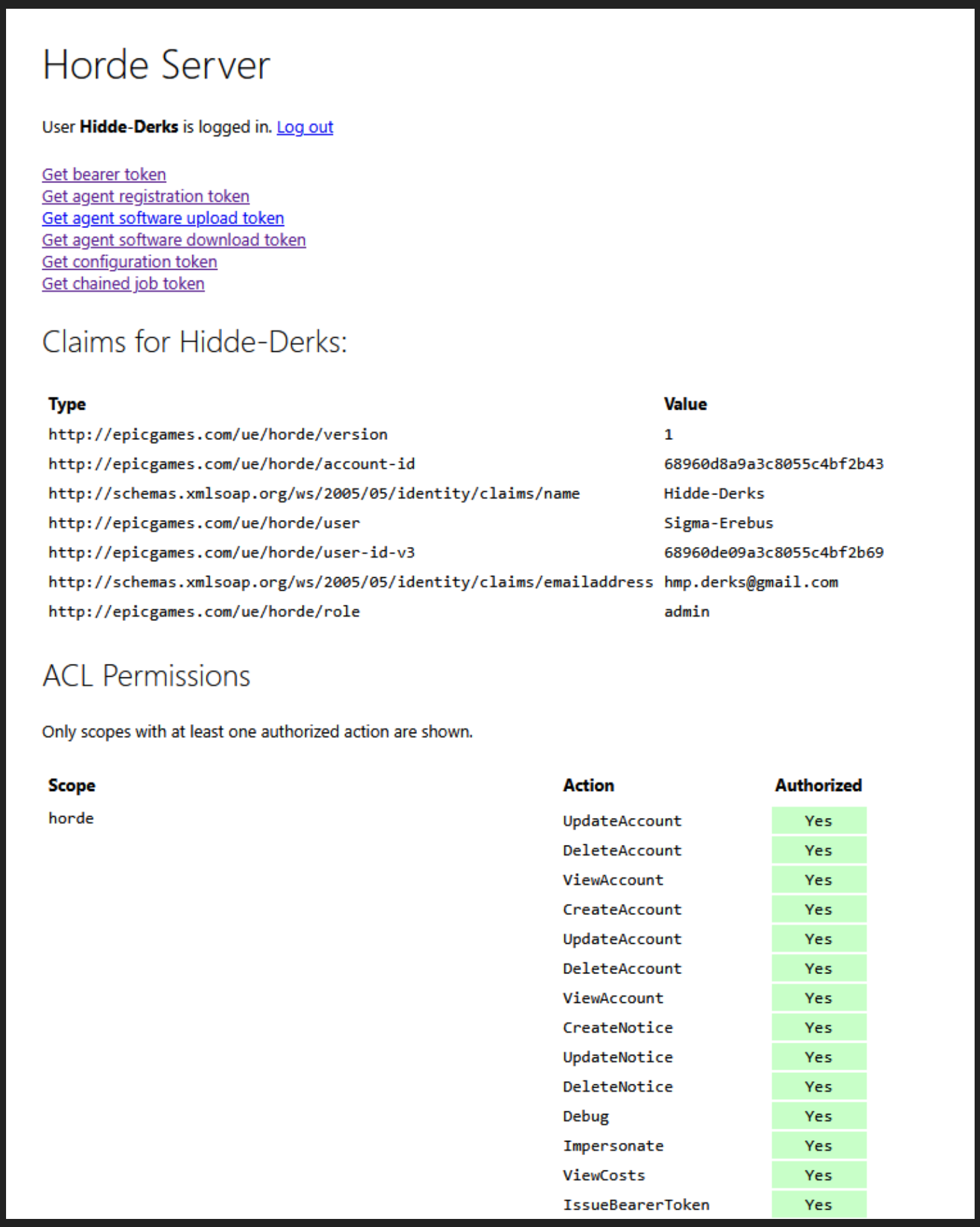
Horde Agent (Windows)

Before getting started with this guide, you do need to have an existing Horde Server up and running, with both a URL for accessing that server, along with an Admin account that can get an agent software download token & agent registration token if authentication is enabled on said server). This guide will purely be for a compute agent, and will not be configured for adding new compute tasks to the cluster.

Step 1.

Navigate to your Horde Server URL, usually in the format of:
`http://<IP>:<Port>`
e.g.
`http://192.168.0.204:13340`
`http://localhost:13340`
`https://192.168.0.204:13340`

This might require a login to the server, but once you're in the dashboard, you'll want to navigate to `{HordeURL}/account` (e.g. `http://192.168.0.204:13340/account`). In this view you can also easily view the exact permissions that your current account has (or has not) got. This is also where we will get our agent software download token & agent registration token:



Horde Server

User **Hidde-Derks** is logged in. Log out

Get bearer token
Get agent registration token
Get agent software upload token
Get agent software download token
Get configuration token
Get chained job token

Claims for Hidde-Derks:

Type	Value
http://epicgames.com/ue/horde/version	1
http://epicgames.com/ue/horde/account-id	68960d8a9a3c8055c4bf2b43
http://schemas.xmlsoap.org/ws/2005/05/identity/claims/name	Hidde-Derks
http://epicgames.com/ue/horde/user	Sigma-Erebus
http://epicgames.com/ue/horde/user-id-v3	68960de09a3c8055c4bf2b69
http://schemas.xmlsoap.org/ws/2005/05/identity/claims/emailaddress	hmp.derks@gmail.com
http://epicgames.com/ue/horde/role	admin

ACL Permissions

Only scopes with at least one authorized action are shown.

Scope	Action	Authorized
horde	UpdateAccount	Yes
	DeleteAccount	Yes
	ViewAccount	Yes
	CreateAccount	Yes
	UpdateAccount	Yes
	DeleteAccount	Yes
	ViewAccount	Yes
	CreateNotice	Yes
	UpdateNotice	Yes
	DeleteNotice	Yes
	Debug	Yes
	Impersonate	Yes
	ViewCosts	Yes
	IssueBearerToken	Yes

we'll want to store these tokens somewhere temporarily for now, as we'll need them later.

Step 2.

Now we can start preparing the docker container. For this, create a folder somewhere on your drive that we can use as a project folder, with the following files:

- *.../Path/To/Root*
  - `Dockerfile`
  - `docker-compose.yaml`
  - `agent.json`
  - `appsettings.json`
  - `appsettings.Production.json`
  - `entrypoint.sh`
  - `horde-api-key.txt`

Note, `Dockerfile` in this instance is a file, not a folder

Paste the agent software download token from step 1 into the `horde-api-key.txt` file. As we'll load that as a docker secret in a bit.

For the other files, give them the following contents and make sure to tweak them to your requirements where relevant (see comments within the files for details):

```
FROM debian:stable

ARG HORDE_SERVER_URL

# Install dependencies
RUN apt-get update && apt-get install -y \
    wine \
    xvfb \
    curl \
    unzip \
    sudo \
    ca-certificates

RUN curl https://packages.microsoft.com/config/debian/12/packages-microsoft-
RUN dpkg -i packages-microsoft-prod.deb
RUN rm packages-microsoft-prod.deb

RUN mkdir -p /home/horde-docker/Horde/Agent && \
    mkdir -p /home/horde-docker/Horde/Logs && \
    mkdir -p /home/horde-docker/Horde/Workspace

RUN --mount=type=secret,id=horde_api_key \
    export HORDE_API_KEY=$(cat /run/secrets/horde_api_key) && \
    curl ${HORDE_SERVER_URL}api/v1/agentsoftware/default/zip -o /home/horde-
RUN unzip /home/horde-docker/Horde/HordeAgent.zip -d /home/horde-docker/Horc

RUN apt-get update && apt-get install -y \
    dotnet-runtime-8.0

RUN groupadd --system --gid 568 horde-docker && \
    useradd --system --gid horde-docker --uid 568 --home-dir /home/horde-doc
    adduser horde-docker sudo && \
    chown -R horde-docker:horde-docker /home/horde-docker

USER horde-docker

COPY entrypoint.sh /entrypoint.sh

WORKDIR /home/horde-docker/Horde

ENTRYPOINT [ "/entrypoint.sh" ]
```

Dockerfile

```yaml
services:
  horde-agent:
    build:
      context: .
      secrets:
        - horde_api_key
      args:
        HORDE_SERVER_URL: https://horde.example.com/
    networks:
      - horde-network
    environment:
      HORDE_SERVER_URL: https://horde.example.com/
      HORDE_DIRECTORY: /home/horde-docker/Horde/Agent
    volumes:
      - ./agent.json:/home/horde-docker/Horde/Agent/Data/agent.json
      - ./appsettings.Production.json:/home/horde-docker/Horde/Agent/appsett
      - ./appsettings.json:/home/horde-docker/Horde/Agent/appsettings.json
    tty: true
    stdin_open: true
    restart: unless-stopped
secrets:
  horde_api_key:
    file: ./horde_api_key.txt
networks:
  horde-network:
    driver: bridge
```

docker-compose.yaml

```bash
#!/bin/bash

cd Agent
dotnet HordeAgent.dll
```

entrypoint.sh

> The files after this point are the same as on a regular Windows deployment

```jsonc
{
    "horde": {
        // if unused will default to the computer's name, but can be used to
        "name": "Docker-Agent",
        "wineExecutablePath": "/usr/bin/wine",
        "ephemeral": true,
        "workingDir": "/home/horde-docker/Horde/Workspace",
        "logsDir": "/home/horde-docker/Horde/Logs",
        // the IP on which other agents can reach this agent (host machine,
        "computeIp": "192.168.0.21",
        "computePort": "7000",
        "properties": {
            "additionalProperty": "value"
        }
    }
}
```

agent.json

```
{
  "Horde": {
    "Server": "ExampleName",
    "ServerProfiles": [
      {
        "Name": "ExampleName",
        "Url": "https://horde.example.com",
        "Environment": "Production",
        "Token": "<AGENT REGISTRATION TOKEN>" // Put your Agent Registratior
      }
    ]
  },
  "Serilog": {
    "MinimumLevel": {
      "Default": "Information",
      "Override": {
        "Microsoft": "Information",
        "System.Net.Http.HttpClient": "Warning",
        "Microsoft.AspNetCore.Routing.EndpointMiddleware": "Warning",
        "Microsoft.AspNetCore.Authorization.DefaultAuthorizationService": "W
        "HordeServer.Authentication.HordeJwtBearerHandler": "Warning",
        "HordeServer.Authentication.OktaHandler": "Warning",
        "Microsoft.AspNetCore.Hosting.Diagnostics": "Warning",
        "Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker": '
        "Serilog.AspNetCore.RequestLoggingMiddleware": "Warning",
        "Grpc.Net.Client.Internal.GrpcCall": "Warning",
        "EpicGames.Horde.Storage": "Debug"
      }
    }
  }
}
```

appsettings.json

```
{

}
```

appsettings.Production.json

the last of these is purely used as a placeholder, having it configured for if it will be needed in future to distinguish Production specific settings (separate from for example an `appsettings.Development.json file`)

Step 3.

To start creating the server container package, first navigate to the project folder using command prompt or a terminal:

```
cd .../Path/To/Root
```

Once there, we can hand the compose file to docker compose, which will build the services.

```
docker compose build
```

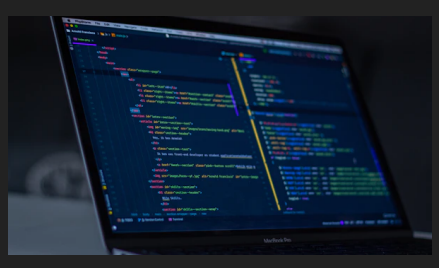
Now that the image has been built, we can start using the image by pulling up (running) the image. Or distribute the image to a registry for usage on multiple other machines (instead of having to build the image on each machine individually).

```
docker compose up
```

Member discussion

0 comments

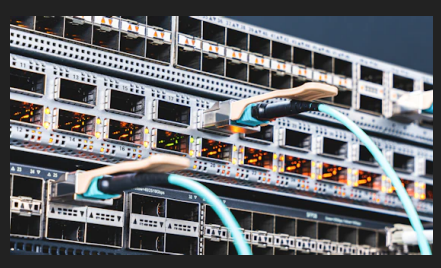
READ MORE



Unreal Horde Agent Deployment (Windows)

Initially created for the Breda University of Applied Scienc...

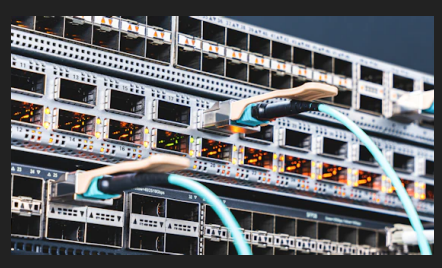
By Hidde Derks — 02 Dec 2025



Unreal Horde Server Deployment (Docker)

Initially created for the Breda University of Applied Scienc...

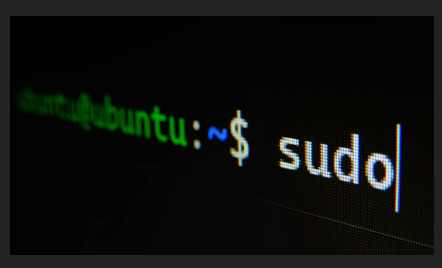
By Hidde Derks — 02 Dec 2025



Unreal Horde Server Deployment (Windows)

Initially created for the Breda University of Applied Scienc...

By Hidde Derks — 02 Dec 2025



Use Powershell to run a command as another user

In Jenkins, every now and then you might need to work...

By Hidde Derks — 25 Jun 2025

Development Blog Hidde Derks

Development blog of Hidde Derks (a.k.a. Sigma-Erebus)

jamie@example.com Subscribe